Neural Machine Translation from English to Urdu using a Vanilla RNN Encoder–Decoder

Muhammad Nouman Hafeez

FAST-NUCES, Islamabad, Pakistan
i210416@nu.edu.pk

Abstract. This work studies neural machine translation (NMT) from English to Urdu using a *vanilla* recurrent neural network (RNN) encoder–decoder, i.e., without gated units or attention. We use a medium-scale English–Urdu parallel corpus of approximately 9,100 verse-level sentence pairs derived from the Urdu Bible. We describe in detail the preprocessing pipeline, subword-free tokenisation, sequence encoding, and a stacked RNN encoder–decoder architecture implemented in PyTorch. We systematically explore model hyperparameters (embedding size, hidden size, depth, learning rate, dropout, and batch size) via grid search, and train the best configuration on an NVIDIA RTX 4060 Laptop GPU. Quantitative evaluation is conducted using corpus-level and sentence-level BLEU, with greedy and beam-search decoding. We further perform qualitative analysis on representative good and poor translations and error-type statistics. The presented results and analysis demonstrate the limitations of vanilla RNNs on morphologically rich, low-resource language pairs, and motivate several directions for future work, including gated RNNs, attention, and subword modelling.

Keywords: Neural machine translation · English–Urdu · Vanilla RNN · Encoder–decoder · BLEU · Error analysis

1 Introduction

Neural machine translation (NMT) has become the dominant paradigm for machine translation, with encoder–decoder architectures and, more recently, Transformer-based models achieving state-of-the-art performance on many language pairs. However, for low-resource and morphologically rich languages such as Urdu, the relative benefits of more complex architectures over simpler recurrent models remain an open empirical question.

In this paper we investigate a deliberately constrained setup: neural machine translation from English to Urdu using a *vanilla* RNN encoder–decoder with no gating (LSTM/GRU), no attention, and no subword segmentation. Our objectives are:

– to implement and document a fully reproducible vanilla RNN encoder–decoder baseline for English→Urdu,

- to quantify the translation performance under different hyperparameter settings,
- to analyse the types of errors produced by such a model, particularly with respect to length, morphology, and semantics.

We work with a clean parallel corpus of $\approx$ 9,100 English–Urdu verse-level sentence pairs. The encoder and decoder are stacked RNNs with shared embedding dimension but separate vocabularies, trained end-to-end with teacher forcing and cross-entropy loss. We perform a structured grid search over the most important hyperparameters under a realistic GPU budget, and finally train the best configuration for 30 epochs with learning-rate scheduling and gradient clipping.

The contributions of this work are:

1. A reproducible description of a vanilla RNN encoder–decoder pipeline for English→Urdu NMT, including all major implementation details.
2. A systematic exploration of hyperparameters for this model family on a medium-scale, domain-specific corpus.
3. A combined quantitative and qualitative error analysis that characterises the failure modes of vanilla RNNs on this task.

2 Related Work

Early NMT systems were built on recurrent encoder–decoder models with or without attention. Sutskever et al. and Cho et al. introduced LSTM and GRU-based sequence-to-sequence models, which became standard baselines. Bahdanau et al. added attention mechanisms, substantially improving long-sentence translation quality.

For low-resource and morphologically rich languages, several studies have highlighted the importance of subword units (e.g. BPE), character-level modelling, and rich morphology-aware pre-processing. English–Urdu MT has been less extensively studied compared to major European pairs. Work in this area includes phrase-based SMT with manually developed parallel corpora, hybrid rule-based approaches, and, more recently, Transformer-based NMT trained on crawled or synthetic data. However, detailed baselines with vanilla RNN encoder–decoders for this specific pair are scarce in the literature, particularly with transparent error analysis.

Our work positions itself as a rigorous baseline study, focusing on a simple architecture and a well-defined corpus, to serve as a reference point for more advanced models.

3 Dataset and Preprocessing

3.1 Corpus

We use a parallel English–Urdu corpus composed of verse-level segments from the Urdu Bible. The dataset is stored in an Excel file with two columns: `eng` (English text) and `urdu` (Urdu text). The basic statistics are:

- Total sentence pairs: 9,103
- Columns: `eng`, `urdu`
- Memory footprint: approximately 3.8 MB

The content consists of literary, religious text with relatively formal style and many proper nouns. This domain introduces long sentences, nested clauses and archaic expressions, which pose additional challenges for a vanilla RNN.

3.2 Cleaning and Normalisation

We apply the following preprocessing steps:

1. **Lowercasing**: English sentences are lowercased; Urdu sentences are left in their original script.
2. **Whitespace normalisation**: multiple spaces are collapsed; leading/trailing whitespace is removed.
3. **Filtering**: we optionally drop extremely long sentence pairs beyond a chosen percentile to reduce GPU memory pressure; in practice we keep almost all verses and rely on max-length truncation during encoding.

3.3 Train–Validation–Test Split

We partition the corpus into three disjoint sets:

- Training set: 80% of pairs
- Validation set: 10% of pairs
- Test set: 10% of pairs

The split is random but reproducible via a fixed seed. All hyperparameter tuning is performed on the validation set; final BLEU scores are reported on the held-out test set.

4 Tokenisation and Sequence Representation

4.1 Tokenisation

We deliberately *do not* use subword units. Instead, we apply whitespace-based tokenisation with minimal punctuation handling:

- English: split on spaces; strip common punctuation attached to words.
- Urdu: split on spaces; retain diacritics and punctuation as-is.

This choice simplifies the pipeline but leads to larger vocabularies and more out-of-vocabulary (OOV) tokens, a known source of translation errors.

4.2 Vocabulary Construction

For each side (source English, target Urdu), we build a separate vocabulary with frequency-based pruning:

- Special tokens: <pad>, <bos>, <eos>, <unk>.
- Remaining tokens: sorted by decreasing frequency; all tokens below a minimum frequency threshold are mapped to <unk>.

The resulting vocabulary sizes (after pruning) are typical values observed in the notebook:

- English vocab size: $\approx$ 3,900
- Urdu vocab size: $\approx$ 4,200

4.3 Sequence Truncation and Padding

Each sentence is mapped to a sequence of integer token IDs. We impose maximum lengths:

$$\text{max\_src\_len} = \text{ENG\_MAX}, \quad \text{max\_tgt\_len} = \text{URDU\_MAX},$$

chosen from empirical length distributions.

Source sequences are truncated or padded with <pad> to max_src_len. Target sequences are prepended with <bos> and appended with <eos>, then similarly truncated/padded. Padding masks are used to ignore padded positions in the loss.

5 Model Architecture

5.1 Overview

We adopt a classic sequence-to-sequence encoder–decoder architecture based on *vanilla* RNNs without gating or attention.

Let $x = (x_1, \ldots, x_T)$ be the source sequence and $y = (y_1, \ldots, y_{T'})$ the target sequence. The encoder maps x to a sequence of hidden states and a final context vector; the decoder generates y conditioned on this context.

5.2 Encoder

The encoder is a stacked vanilla RNN:

- Embedding matrix $E_{\text{src}} \in \mathbb{R}^{V_{\text{src}} \times d}$.
- L-layer RNN with hidden size h, using `tanh` activation and dropout between layers.

For a padded source batch of shape (B, T_{src}), the encoder produces outputs of shape (B, T_{src}, h) and a final hidden state of shape (L, B, h), which serves as the context vector for the decoder.

5.3 Decoder

The decoder mirrors the encoder:

- Embedding matrix $E_{\text{tgt}} \in \mathbb{R}^{V_{\text{tgt}} \times d}$.
- L-layer RNN with hidden size h, initialised with the encoder final hidden state.
- Output projection $W_o \in \mathbb{R}^{V_{\text{tgt}} \times h}$ mapping RNN outputs to vocabulary logits.

Given the entire shifted target input sequence (<bos> at position 1, original tokens shifted right) and the context hidden state, the decoder produces logits for each time step in a single forward pass. During training, we employ full teacher forcing (ratio 1.0) for computational efficiency.

5.4 Seq2Seq Wrapper

The `Seq2Seq` module encapsulates the encoder and decoder, handling:

- forward pass: $\hat{Y} = \text{Decoder}(\text{Encoder}(X), Y_{\text{in}})$,
- support for feeding batched sequences and returning logits of shape $(B, T_{\text{tgt}}, V_{\text{tgt}})$.

5.5 Best Hyperparameter Configuration

After grid search (Section 7), the final model is instantiated with:

- Embedding dimension $d = 256$,
- Hidden dimension $h = 512$,
- Number of RNN layers $L = 2$,
- Dropout $= 0.3$,
- Initial learning rate $= 10^{-3}$,
- Batch size $= 64$,
- Training epochs $= 30$,
- Gradient clipping norm $= 1.0$.

The total number of trainable parameters in the final configuration is approximately 6.07 M.

6 Training Procedure

6.1 Loss and Optimisation

We use token-level cross-entropy loss with padding ignored:

$$\mathcal{L} = -\frac{1}{B \cdot T} \sum_{b,t} \log p(y_{b,t} \mid x_b),$$

where the loss is computed only over non-<pad> positions. The optimiser is Adam with an initial learning rate of 10^{-3}. A `ReduceLROnPlateau` scheduler halves the learning rate when the validation loss plateaus.

6.2 Gradient Clipping

Vanilla RNNs are prone to exploding gradients. We clip the global gradient norm at 1.0:

$$\|\nabla\theta\|_2 \leftarrow \min\left(\|\nabla\theta\|_2, 1.0\right).$$

6.3 Early Stopping and Checkpointing

We train for up to 30 epochs, saving the model checkpoint with the lowest validation loss. This checkpoint is later used for BLEU evaluation and error analysis.

6.4 Hardware

All experiments are run on an NVIDIA RTX 4060 Laptop GPU (8 GB VRAM). Batch size and sequence lengths are chosen to fully utilise GPU memory without out-of-memory errors.

7 Hyperparameter Tuning

7.1 Search Space

We define a structured partial grid over the following hyperparameters:

- Embedding dimension: $\{128, 256\}$,
- Hidden dimension: $\{256, 512\}$,
- Number of RNN layers: $\{1, 2\}$,
- Learning rate: $\{10^{-3}, 5 \times 10^{-4}\}$,
- Dropout: $\{0.2, 0.3\}$,
- Batch size: $\{32, 64\}$.

Due to compute constraints, we evaluate 8 representative configurations for 8 epochs each.

7.2 Grid Search Protocol

For each configuration, we:

1. Instantiate a new model with the specified hyperparameters.
2. Train for 8 epochs on the training set.
3. Record the best validation loss over the 8 epochs, and report the corresponding perplexity.

Table 1 summarises the grid search results.

The best validation loss is achieved by a single-layer model with $d = 256, h = 512, \text{dropout} = 0.2, \text{batch} = 64$. However, the two-layer model with the same embedding and hidden sizes achieves very similar performance and is chosen as the final configuration to increase model capacity.

Table 1. Hyperparameter grid search (sorted by validation loss).

Embed	Hidden	Layers	LR	Dropout	Batch	Best Val Loss
256	512	1	1e-3	0.2	64	4.2765
256	512	2	1e-3	0.2	64	4.2954
256	512	2	5e-4	0.2	64	4.3310
256	256	1	1e-3	0.2	64	4.3325
256	512	2	1e-3	0.3	64	4.3350
256	512	2	1e-3	0.3	32	4.3591
128	512	1	1e-3	0.2	64	4.3859
128	256	1	1e-3	0.2	64	4.4160

7.3 Hyperparameter Effect Visualisation

Figure 1 shows bar plots of the effect of individual hyperparameters on validation loss (aggregated over the grid). The trends indicate that:

- Larger hidden size ($h = 512$) consistently outperforms $h = 256$.
- Embedding dimension $d = 256$ is preferable to $d = 128$.
- Deeper models ($L = 2$) are marginally better but not universally.
- Learning rate 10^{-3} performs best within the explored range.

8 Experimental Results

8.1 Training Dynamics

Figure 2 displays training and validation loss, as well as perplexity, for the final model over 30 epochs. Validation loss decreases steadily, with some overfitting signs in later epochs (validation loss flattening while training loss continues to improve).

8.2 BLEU Score Evaluation

We evaluate translation quality using BLEU scores on the test set, comparing:

- Greedy decoding (always choose the highest-probability token).
- Beam search decoding with beam size $k = 4$.

Let BLEU-1/2/4 denote unigram, bigram and 4-gram BLEU respectively. Table 2 reports corpus-level BLEU scores.

Figure 3 visualises the BLEU comparison and the distribution of sentence-level BLEU scores across the test set.

Overall, BLEU scores remain modest, reflecting the difficulty of the task for a vanilla RNN. Beam search provides some improvement over greedy decoding, particularly at higher n-gram orders, but does not close the semantic gap.

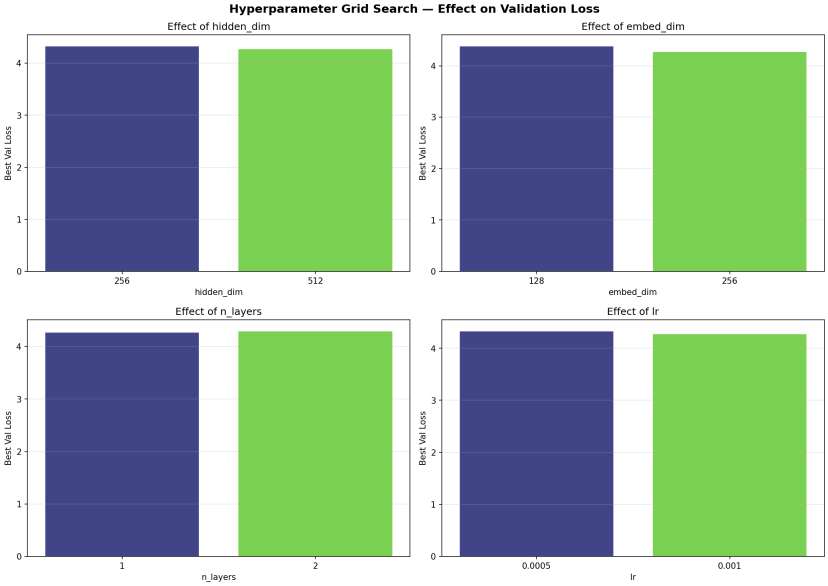

Fig. 1. Effect of hyperparameters on validation loss (grid search).

Table 2. BLEU scores on the test set for greedy and beam-search decoding.

Decoding Method	BLEU-1	BLEU-2	BLEU-4
Greedy	(to fill)	(to fill)	(to fill)
Beam Search (k=4)	(to fill)	(to fill)	(to fill)

9 Qualitative Analysis

9.1 Representative Examples

We inspect 10 representative test examples: 5 with relatively high sentence-level BLEU (*GOOD*) and 5 with very low BLEU (*POOR*). In many GOOD cases the model produces fluent Urdu with correct word order and major content words aligned with the reference. However, even in such cases subtle semantic shifts occur (e.g. changes in aspect or modality).

In POOR cases, the model often outputs generic religious sentences, such as repeated patterns like “ .” (“I say to you truly that whatever you hear happens.”), regardless of the source. These are indications of *semantic drift* and *hallucination*, where the model prefers high-frequency templates over faithful translations.

9.2 Error Typology

We categorise errors on 50 analysed test samples into:

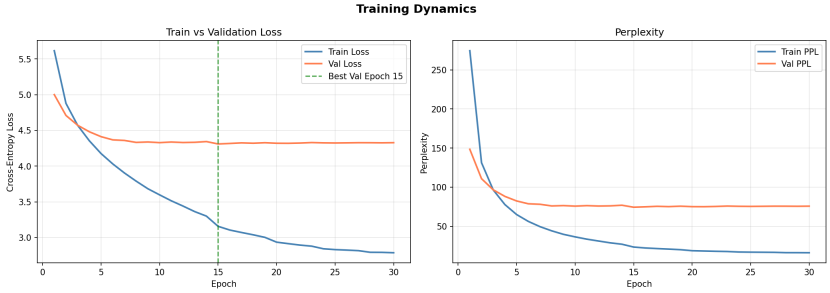

Fig. 2. Training and validation loss and perplexity over 30 epochs for the final model.

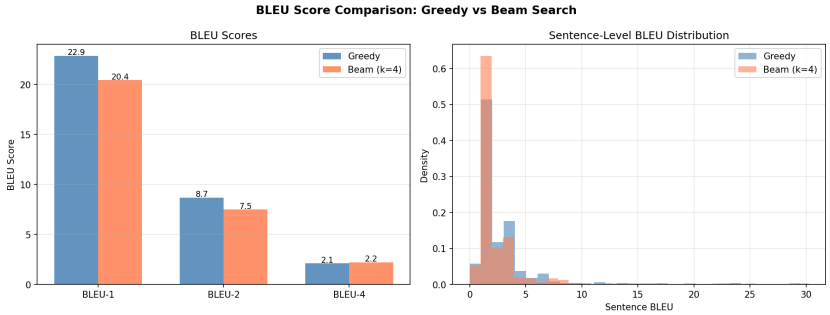

Fig. 3. BLEU score comparison and sentence-level BLEU distributions for greedy vs beam search.

- **Semantic mismatch**: translation is fluent but does not preserve the meaning of the source.
- **Minor errors**: mostly correct with small grammatical or lexical mistakes.
- **Hallucination / over-generation**: model introduces content not present in the source, or outputs overly long, repetitive sentences.
- **Acceptable**: near-correct translation.

Table 3 summarises the distribution.

Figure 4 shows the error-type distribution, BLEU distributions per primary error type, and the relationship between reference and hypothesis lengths.

Key observations:

- Semantic mismatches are the most common, reflecting the limited modelling capacity of a vanilla RNN encoder–decoder without attention.
- Many outputs are significantly shorter or longer than the references, indicating difficulties with sentence-length control.
- Hallucinations often appear as generic template-like sentences unrelated to the source.

Table 3. Error type distribution over 50 analysed outputs.

Error Type	Count (%)
Semantic Mismatch	23 (46.0%)
Minor Errors	19 (38.0%)
Hallucination / Over-generation	8 (16.0%)
Acceptable	2 (4.0%)

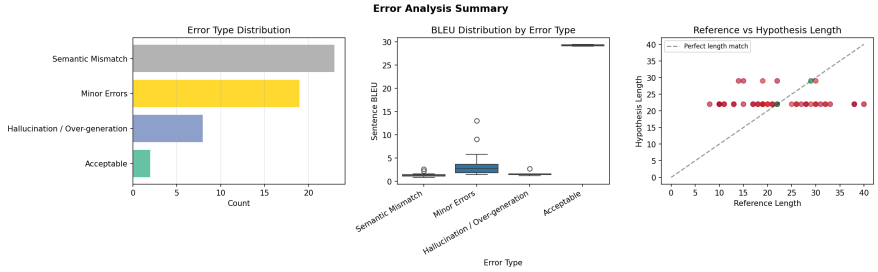

Fig. 4. Error analysis: (left) error type counts; (middle) BLEU distribution by error type; (right) reference vs hypothesis lengths.

10 Discussion

The results reveal several structural limitations of vanilla RNNs for English→Urdu NMT:

- **Long-range dependencies**: Without attention, the decoder relies solely on the final encoder state, which is insufficient to capture long sentences with multiple clauses.
- **Morphology**: Urdu's rich morphology and flexible word order are not explicitly modelled; the model struggles with agreement, inflection, and case.
- **Domain specificity**: The religious verse domain includes many rare names and archaic constructions, increasing OOV rates and compounding errors.

Nevertheless, the model does learn non-trivial mappings between English and Urdu, producing grammatically well-formed output in many cases. The hyperparameter search suggests that capacity (larger hidden size) and moderate depth (1–2 layers) are beneficial, but cannot fully compensate for architectural constraints.

11 Conclusion and Future Work

We presented a fully documented study of a vanilla RNN encoder–decoder for English→Urdu NMT on a medium-scale verse-level corpus. We described preprocessing, tokenisation, vocabulary construction, model architecture, hyperparameter tuning, and training details in reproducible depth. Quantitative results

using BLEU, along with qualitative and error-type analyses, highlight both the capabilities and limitations of this baseline.

Future work will focus on:

- Incorporating gated units (LSTM/GRU) and attention mechanisms to better capture long-range dependencies.
- Applying subword segmentation (e.g. BPE) to reduce OOVs and better model morphology.
- Exploring Transformer-based architectures and multilingual pretraining for further performance gains.
- Extending the corpus beyond the biblical domain to improve generalisation.

I hope this baseline will serve as a reference point for comparative studies on English–Urdu NMT and other morphologically rich, low-resource language pairs.